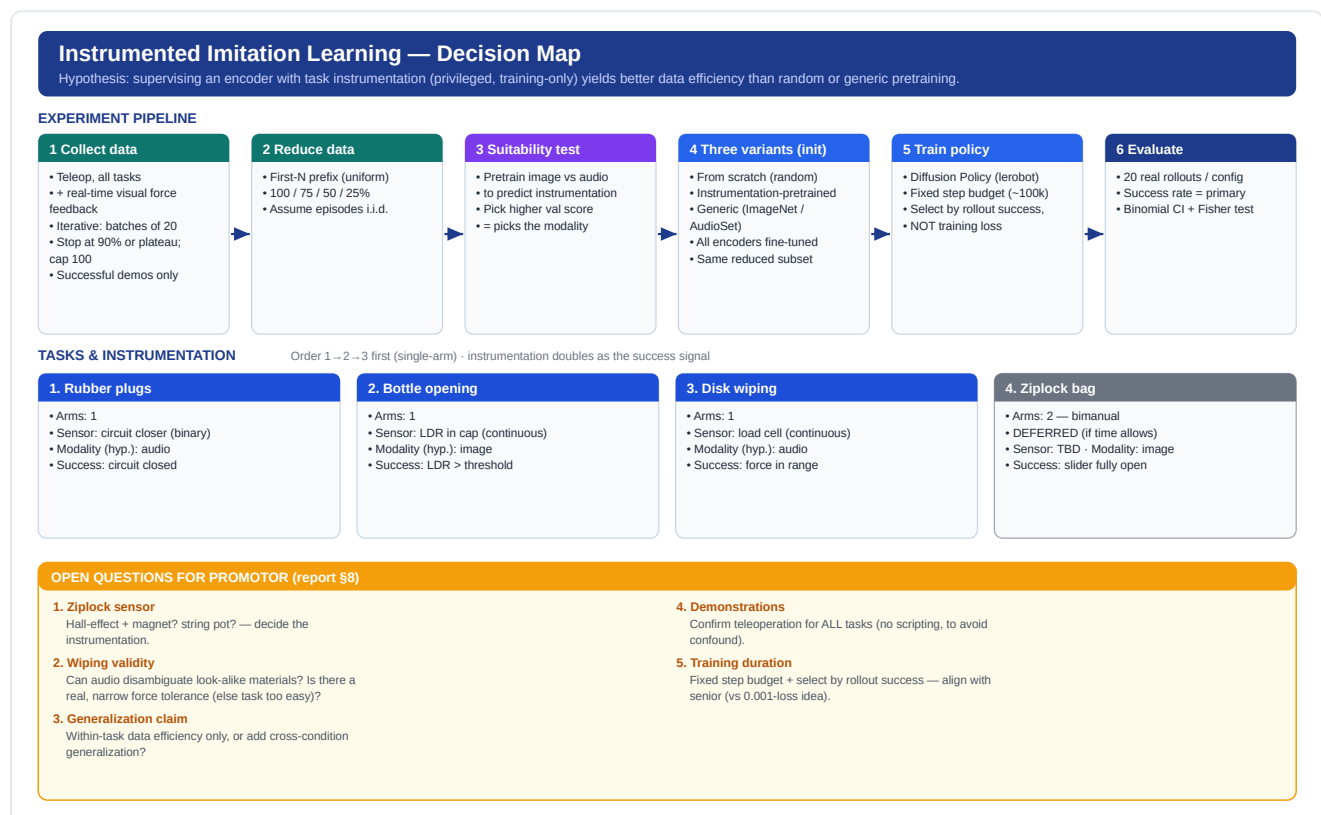


Instrumented Imitation Learning — Experimental Design Report

Prepared for: Promotor meeting **Date:** 1 July 2026 **Status:** Design largely locked; open decisions flagged in §8

At a glance — decision map

One-page overview of the full design and the open questions. A standalone landscape PDF (*Promotor_Meeting_Decision_Map.pdf*) is on the Desktop for screen-sharing during the meeting.



1. Research question & hypothesis

Research question: Does *instrumentation* improve policy performance and data efficiency in imitation learning?

Instrumentation = task-specific auxiliary sensors added to the robot's *environment* that provide **privileged information available during training only, not at inference**. Examples: a rubber plug wired to close an electrical circuit when seated; an LDR inside a bottle cap reporting how open it is.

Refined hypothesis: Supervised pretraining of a perception encoder on these visible-but-labelled instrumentation signals acts as an inductive bias, teaching the encoder to focus on task-relevant features faster than unsupervised/joint learning. The result is **better data efficiency** during policy fine-tuning — the same success rate reached with fewer demonstrations.

The signal does **not** need to be hidden from the camera/microphone. It only needs a label that accelerates representation learning. The instrumentation must be perceivable in the image or audio; otherwise the encoder learns only spurious correlations.

2. Core contribution — the three-variant comparison

The heart of the paper is a controlled comparison of three policies that differ **only in how one encoder is initialized**. Everything else (architecture, data, training steps) is identical.

Variant	Pretrained encoder init	Purpose
From scratch	Random (image + audio)	Lower baseline
Instrumentation-pretrained	One encoder pretrained to predict the instrumentation signal	The proposed method
Generic-pretrained	Same encoder pretrained on generic data (ImageNet / AudioSet)	Controls for "pretraining in general"

Why the generic baseline matters: it isolates the claim. If instrumentation-pretrained beats generic-pretrained, the gain comes **specifically from the task instrumentation**, not from pretraining per se. The generic variant always mirrors the instrumentation variant's encoder choice (same modality), differing only in the pretraining *source*. All encoders are **fine-tuned** during policy training (not frozen) — same steps, same everything, only the initialization differs.

3. Tasks & instrumentation

Ordered simple → complex, single-arm before bimanual. **Start with rubber plugs** to validate the full pipeline (collect → pretrain → train → evaluate).

#	Task	Arms	Instrumentation	Signal	Hypothesized modality*	Success criterion
1	Rubber plugs in metal sheet	1	Metal ring on plug closes a circuit on the sheet (like a button)	Binary	Audio (seating click)	Circuit closed
2	Bottle opening (flick-switch cap)	1	LDR inside the cap	Continuous	Image (visible cap rotation)	LDR > threshold
3	Disk wiping at consistent pressure	1	Load cell under surface (or FSR under disk)	Continuous	Audio (friction sound)	Mean contact force in acceptable range
4	Ziplock bag open/close	2 (bimanual)	TBD — see §8	Continuous	Image (slider position)	Slider fully open

* The hypothesized modality is only a starting guess — the actual choice is made objectively by the suitability metric (§4).

Instrumentation doubles as the success signal for every task — either immediately (plug circuit) or after processing (wiping force). No separate evaluation logic is needed; the sensor that supervises training also defines test-time success.

Ziplock is deferred: it is the only bimanual task (deformable, cannot be done one-armed) and is not directly comparable to the other three. It will be added *later, if time/budget allows*, and requires extending the single-arm framework.

4. Modality selection — the suitability metric

To avoid the "you cherry-picked the modality that worked" critique, the choice of which encoder to pretrain (image vs. audio) is made by a **pre-registered, objective test measured before any policy evaluation**:

1. Split demonstrations 80% train / 20% validation.
2. Pretrain the **image** encoder to predict the instrumentation signal → record validation accuracy / R^2 .
3. Pretrain the **audio** encoder the same way → record validation accuracy / R^2 .
4. **Pick the higher-scoring modality** for that task.

This yields a clean per-task table (Image score vs. Audio score → chosen modality) that justifies each decision on evidence rather than intuition.

5. Encoder pretraining

- **Objective follows the signal type:** classification (BCE) for binary/discrete instrumentation (plugs); regression (MSE) for continuous instrumentation (bottle, wiping).
 - **Normalization:** continuous signals normalized to $[0, 1]$, preferably by **percentile within the demonstration dataset**. The model then learns "*is this force high or low relative to what's typical for this task?*" rather than absolute newtons — which should generalize better across surfaces/materials.
 - **Pretraining data = the actual demonstration data** (not separately collected).
 - **Hyperparameters:** LR $1e-4$, early stopping on validation-loss plateau. Cheap (~10–30 min/task).
-

6. Data-efficiency protocol

- **Data-reduction levels:** 100%, 75%, 50%, 25% of N.
- **Reduction method:** first N episodes (uniform prefix), assuming episodes are i.i.d. — chosen to avoid multiplying the number of runs by a k-repeat factor. (*Optional zero-cost robustness: shuffle with a fixed seed, then take the prefix.*)
- **Both pretraining and fine-tuning use the same reduced subset.** This is the honest data-efficiency test: at 25%, *everything* sees only 25%. If instrumentation still wins, it proves the signal is more informative per sample — not that it secretly saw more data.

Finding N (total episodes) — iterative protocol

1. Collect a batch of 20 demonstrations.

2. Train the from-scratch policy on all episodes collected so far.
3. Evaluate with 20 rollouts.
4. Repeat until success $\geq 90\%$, or a **plateau** is detected ($\leq 5\%$ improvement over two consecutive iterations).
5. **Cap N at 100 episodes** to bound worst-case effort.
6. N is then fixed; run the full experiment at 100/75/50/25% of N.

Successful demos only — failed episodes are discarded during collection (so ~120 collected may be needed to reach ~100 successful). N is task-dependent, which is honest and expected.

7. Training, data collection & evaluation

Training protocol

- **Fixed number of training steps for all variants** (equal compute budget) — chosen over "train to a target loss." Same steps = fair, simple comparison. This matches standard practice for Diffusion Policy (see *Training duration* below).
- Fine-tuning uses identical hyperparameters/duration across variants.
- **Framework:** lerobot (ACT / Diffusion Policy). Note: **Diffusion Policy has no memory**, so the instrumentation signal must be **instantaneous** (current-state force/contact), never cumulative (e.g. wipe coverage is ruled out).

Training duration — resolves the "target-loss" question (literature-grounded)

The senior's suggestion to *train to a target loss ≈ 0.001 and reuse that step count* does not transfer to Diffusion Policy. Literature check:

- **DP's training loss is not a convergence/quality signal.** It is the denoising MSE on the predicted noise ϵ , averaged over random diffusion timesteps (`num_train_timesteps=100` , `prediction_type=epsilon`). Because that average includes high-noise steps where the noise is essentially unpredictable, the loss **plateaus at a setup-dependent value and never decays toward ~ 0.001** . A 0.001 target is closer to an **ACT-style L1 action-reconstruction loss** — a different quantity. DP's absolute loss also shifts with noise schedule, prediction type, and action normalization, so it is not comparable across runs.
- **The field trains to a fixed budget, not a loss.** The original DP repo uses a fixed `num_epochs : 600` (real-robot image, UNet), **3050** (sim image, e.g. PushT/robomimic hybrid), up to **5000–8000** (lowdim / harder sim). lerobot (our framework) defaults to **steps = 100 000** with periodic evaluation every 20k steps.
- **Checkpoints are selected by rollout success, explicitly not by loss.** DP configs set `checkpoint.topk.monitor_key = test_mean_score ( mode: max )`, evaluated every 50 epochs. Chi et al. (RSS 2023) state that low validation loss does not reliably track task success.

Decision: Fixed step budget for all variants — pick a count where the **from-scratch variant at 100% data clearly plateaus in rollout success** (checked via periodic rollout eval), then reuse that same budget across all variants and data levels. Grounded anchor to start from: **lerobot's 100k-step default** (or DP's 600 epochs for real-robot image tasks).

Sources: `real-stanford/diffusion_policy` config files (`num_epochs` , `checkpoint.topk.monitor_key = test_mean_score`); `huggingface/lerobot`

`configs/train.py ( steps: int = 100_000 );` Chi et al., Diffusion Policy: Visuomotor Policy Learning via Action Diffusion, RSS 2023.

Data collection

- **Teleoperation for all tasks** (not scripted) — one consistent method avoids confounding task type with data-collection method, and gives ecological validity + trajectory diversity.
- **Real-time visual force feedback during teleop** (on-screen bar/needle with a target band) so the operator applies *intentional* expert force. Without feedback, recorded force is incidental and noisy.

Evaluation

- **20 real-world rollouts per configuration.**
- **Primary metric: success rate** (secondary metrics dropped as non-essential — the success-rate curve is the story).
- **Rollout timeout:** 2× median demonstration duration → mark as failure.
- **Statistics:** 95% binomial confidence intervals per success rate; Fisher's exact test for pairwise variant comparisons at each data level.

Experimental scale (3 single-arm tasks first)

- 3 tasks × 3 variants × 4 data levels = **36 policy trainings** and **36 configs × 20 = 720 rollouts** (~36 h hands-on).
- Adding bimanual ziplock later = +240 rollouts + framework extension.

Hardware: 2× UR5 arms with Robotiq + Schunk grippers. Codebase: `robot-imitation-glue` (currently single-arm + gripper; needs extension for the bimanual ziplock task).

8. Open questions to resolve in the meeting

These are the decisions I deliberately deferred to you, promotor:

1. **Ziplock-bag instrumentation.** No sensor decided yet. Options considered: Hall-effect sensor + magnet on the slider (recommended: minimal intrusion, continuous), string potentiometer, or capacitive distance sensor. A linear potentiometer was ruled out (too rigid, changes bag dynamics). *Decision needed.*
2. **Wiping-task validity & force disambiguation.**
 - If two surfaces are **visually identical but need different forces**, can **audio** (sliding-friction sound) disambiguate the required force? Needs a suitability check across materials.
 - **Force tolerance:** if *any* force across a wide range works equally well, the task is too easy and unsuitable for imitation learning. We must confirm a meaningful, narrow force tolerance exists.
3. **Generalization claim.** Current scope = **within-task data efficiency only** (same objects/setup, less data). Should we also promise **cross-condition generalization** (new materials, new bottles)? That roughly doubles the experiments. Is there pressure/expectation for it?
4. **Scripted vs. teleoperated demonstrations.** Chosen: teleop for all (consistency). But deformables (ziplock) are hard to script, and mixing would confound results — so it must be **all-or-none**. Confirm teleop-everywhere is acceptable.
5. **Training-duration criterion (align with senior).** Senior suggested training to a **target loss ≈ 0.001** and reusing that step count. **Resolved via literature (see §7, Training duration):** target-loss is not a

valid anchor for Diffusion Policy — the field trains to a **fixed step budget** and selects checkpoints by **rollout success, not loss**. Proposed decision: fixed budget (start from lerobot's 100k-step default), sized so the from-scratch variant plateaus in rollout success. *Point to align on with the senior.*

9. One-paragraph summary of the method

For each manipulation task we equip the environment with a task-specific sensor providing privileged information during training only. Guided by an objective suitability metric, we pretrain either the image or audio encoder to predict this instrumentation signal, then fine-tune the full policy. We compare three initializations — random, instrumentation-pretrained, and generic-pretrained (ImageNet/AudioSet) — across four data-reduction levels (100/75/50/25%), evaluating each with 20 real-world rollouts. If the instrumentation-pretrained encoder consistently reaches higher success with fewer demonstrations than the generic-pretrained one, it shows that task-specific instrumentation — not pretraining in general — drives the data-efficiency gain.

10. Next concrete steps

1. Resolve the §8 items with the promotor.
2. Build instrumentation hardware for plugs, bottle, and wiping.
3. Implement the suitability-metric pipeline.
4. Run the rubber-plug task end-to-end first to validate the pipeline before scaling to all tasks.